



3 SHAKE

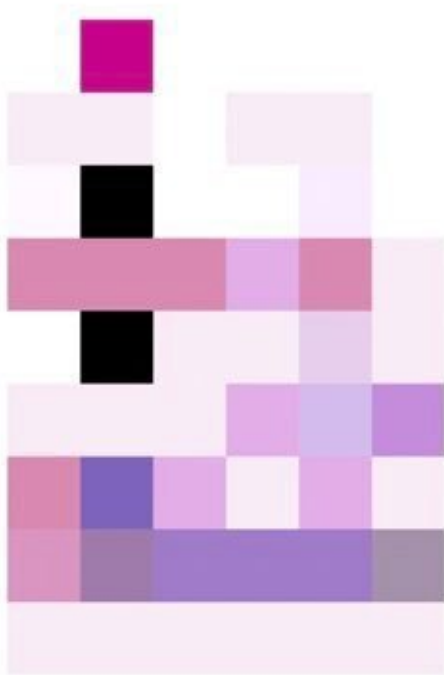
30分でわかる

Secure APIs

書籍の全体像を掴む

2026/XX/XX 3-shake SRE Tech Talk

@nwiizo 30min



nwiizo

株式会社スリーシェイクでプロのソフトウェアエンジニアをやっているものです。格闘技、読書、グラビアが趣味でよく本を紹介しています。

技術書翻訳を手がけるたび、わかることが1つ増えるのと引き換えに、わからないことが3つ増えていく。

インターネット上では **nwiizo** を名乗り、ブログ「**じゃあ、おうちで学べる**」を運営しています。X / GitHub もこのIDでやっています。

about 3-shake



Mission

インフラをシンプルにしてイノベーションが起こりやすい世界を作る

Vision

労苦（Toil）を無くすサービスを適正な価格で提供し続ける

SRE



スリーク

日本のSREをリードする

SRE総合支援からセキュリティ対策を全方位支援

Data Engineering



レコナー

**あらゆるサービスを
連携するハブになる**

クラウド型ETL/データパイプ
ラインサービスの決定版

Security



セキュリファイ

**事業者が抱える
セキュリティリスクを無くす**

ワンストップで実現する
セキュリティ対策

HR (Engineer Hiring)



リランス

**良いエンジニアに
良い案件を**

フリーランスエンジニアに
「今よりいい条件」を



Sreakeのお仕事

クラウドネイティブなアプローチで、お客様の事業をより安全に、競争力のあるサービスへ

提供サービス

SRE/DevOps支援

- Kubernetes構築・運用
- クラウドネイティブ化推進
- Observability導入

セキュリティ支援

- セキュア・バイ・デザイン
- ゼロトラスト導入
- 脆弱性診断・改善支援

データ活用支援

- データ基盤構築
- BigQuery/Snowflake
- 分析基盤最適化

ご依頼・ご相談お待ちしております

<https://sreake.com/>



この発表で解決できること

こんな悩みを持っていませんか？

「APIセキュリティは認証・認可だけと思っていた」

→ APIセキュリティは設計・実装・インフラの3軸にまたがる包括的な分野。OWASP API Top 10の脆弱性を体系的に整理します

「リリース前にチェックリストで確認している」

→ 後付けセキュリティの限界を知り、シフトレフト・ゼロトラスト・セキュア・バイ・デザインを実装する道筋を提示します

「内部APIだから保護しなくていい」

→ 「内部APIというものは存在しない」という原則を、ゼロトラストの考え方とともに学びます

今日の30分で「この本、読んでみよう」と思えるようになること

※30分で12章を扱う以上、各手法の実装詳細は書籍本体に委ねます



書籍の概要

Secure APIs

- 著者: Jose Haro Peralta
- 翻訳: 株式会社スリーシェイク
- 原書: Manning Publications

本書の一言要約

APIセキュリティを「認証・認可の話」から「設計・実装・インフラを貫く包括的な実践」へ拡張する実装ガイド。OWASP API Top 10の脆弱性を起点に、シフトレフト・ゼロトラスト・セキュア・バイ・デザインを具体的なコード・アーキテクチャに落とし込む。



本日の流れ

1. **Part 1: 原則** — APIセキュリティとは何か (Ch.1-3)

2. **Part 2: 脆弱性** — 主要な攻撃ベクトル (Ch.4-5)

3. **Part 3: 設計と実装** — セキュア・バイ・デザイン (Ch.6-9)

4. **Part 4: 高度な領域** — FAPI・観測・テスト (Ch.10-12)

原則 → 脆弱性 → 設計と実装 → 高度な領域 の流れ



書籍は12章で構成されている

Part	テーマ	章	主要な概念
1. 原則	Why & 基礎	Ch.1-3	セキュア・バイ・デザイン、シフトレフト、ゼロトラスト
2. 脆弱性	OWASP API Top 10	Ch.4-5	BOLA, BFLA, 認証不備, SSRF, リソース消費, 設定ミス
3. 設計と実装	How	Ch.6-9	安全な設計、JWT/OAuth/OIDC、認可ミドルウェア、APIゲートウェイ
4. 高度な領域	応用	Ch.10-12	FAPI、オブザーバビリティ、セキュリティテスト

原則を理解 → 脅威を知る → 安全に設計・実装 → 運用で守り続ける



Part 1: 原則

Ch.1-3

最初の問い：「APIセキュリティとは認証・認可だけなのか？」

Ch.1 APIセキュリティとは

定義：APIの脆弱性を研究し、検出・防止する分野。 単なる認証・認可ではない。設計・実装・インフラの全層にまたがる包括的なアプローチを必要とする。

数字で見るAPIセキュリティの現在地

データ	出典
インターネットトラフィックの57～83%がAPI	Cloudflare 57% / Akamai 83%
2023年Q1のAPI攻撃は前年比400%増	Salt Security 2023 Q1レポート
2022年の世界全体のAPI攻撃試行は90億回超	Akamai
API侵害1件あたりの平均被害額 610万ドル	Kong

APIはシステムのゲートウェイ。ドアを開けるなら、鍵をかける方法を知らねばならない



APIセキュリティの3つの軸

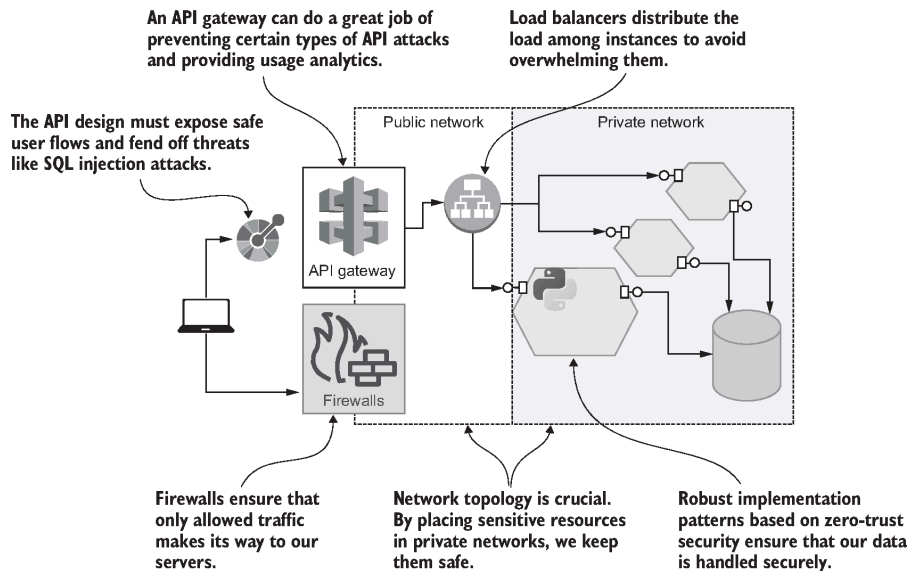


図1.1 APIセキュリティの構成要素 より引用

APIセキュリティは「設計」「実装」「インフラ」の3軸を同時に考える必要がある。どこか1つだけ強化しても、他の軸の弱点から侵入される。

設計

安全なユーザーフロー、データモデル、認可境界。SQLインジェクションのような脅威に防御を組み込む。

実装

データ検証、エラー処理、ライブラリ選定、依存関係。シークレット管理、暗号化。

インフラ

APIゲートウェイ、ファイアウォール、ロードバランサー、ネットワークセグメンテーション、WAF。



「ゲートウェイで守ればいい」「実装はライブラリ任せ」ーこの発想がAPI侵害の根本原因

Ch.2 組織とAPIセキュリティの整合

APIセキュリティは技術問題であると同時に組織問題。 ステークホルダーがその重要性を理解し、作業時間を割り当てなければ、いくら技術的に正しくても実装されない。データ侵害が起きるまで表面化しないという厄介さがある。

組織で動かすための3ステップ

ステップ	やること
1. 評価	現在のセキュリティ・ポスチャーを把握。NIST/CIS/ISOフレームワーク参照
2. 脅威モデリング	チームスポーツとして実施。STRIDEで脅威カテゴリを整理
3. プログラム化	継続的なセキュリティチェックを開発プロセスに組み込む

「監査が来たから慌てて対応」では遅い。今すぐ評価を始める



Ch.3 シフトレフトAPIセキュリティ

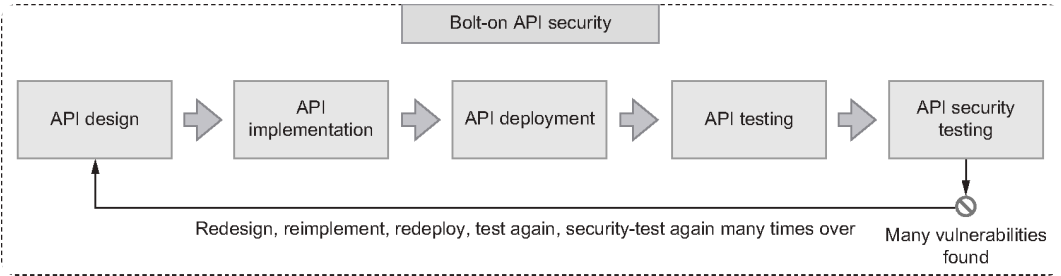


図3.1 SDLCの最後にAPIセキュリティテストを実行する従来アプローチ より引用

従来：完成したAPIにセキュリティテストを後付け（ボルトオン）
→ 表面的な脆弱性しか見つけれられない。

シフトレフト：設計段階からセキュリティを組み込み、開発サイクル全体で検証する。

Before / After

観点	ボルトオン（従来）	シフトレフト（本書の推奨）
セキュリティ対応のタイミング	リリース直前のテストフェーズ	設計・実装・デプロイの全段階
発見できる脆弱性	SSL欠如、XSS、表面的な脆弱性	ビジネスロジック、アクセス制御の不備
修正コスト	高（再設計・再実装が必要）	低（設計時に潰せる）
フィードバックループ	数週間～数ヶ月	数分～数日

API侵害の多くはビジネスロジックの欠陥。表面検査だけでは見つからない



Ch.3 ゼロトラストAPI

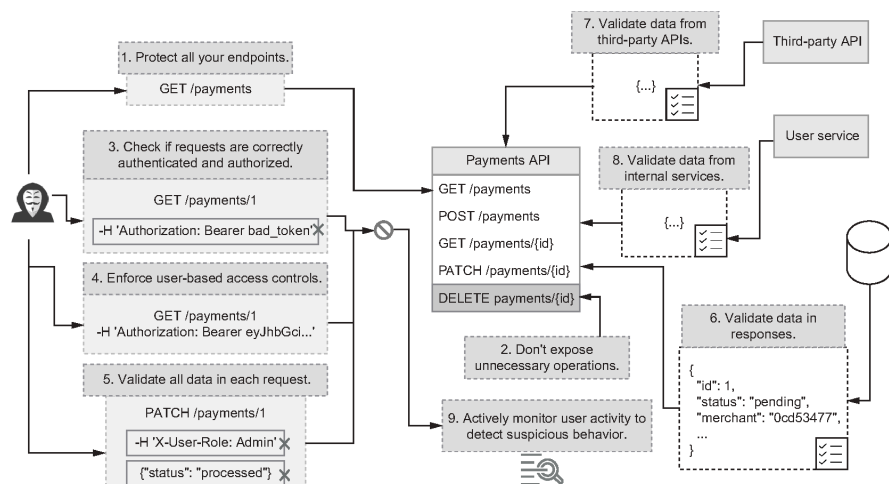


図3.8 NIST 800-207の原則を適用するゼロトラストAPI より引用

ゼロトラスト = 「信頼するな、検証せよ」。NIST 800-207に基づくモデル。すべてのリクエスト・データソース・認証情報を、起源に関係なく検証する。

4つの実践原則

すべてを検証する

入力、トークン、レスポンス、サードパーティAPI。データの起源を信頼しない。

内部APIも保護する

「内部だから安全」は幻想。横展開（lateral movement）への耐性を持つ。

知らないものは守れない

シャドウAPI・ゾンビAPI・ゴーストAPIを発見し、カタログ化する。

DevSecOps

CI/CDにセキュリティを統合し、自動的に検証する。

「内部APIというものは存在しない」 — これがゼロトラストの第一原則



原則はわかった。次は具体的な脅威

Part 1でセキュア・バイ・デザイン、シフトレフト、ゼロトラストという3つの原則を見てきました。原則は明確になった。

しかし、原則だけでは「具体的に何を防ぐか」が分からない。Part 2では、OWASP API Security Top 10に基づく**主要な脆弱性**を扱います。実際の攻撃ベクトルを知ること、設計・実装の指針が具体になります。



Part 2: 脆弱性

Ch.4-5

次の問い：「実際にどんな脅威があるのか？」

OWASP API Security Top 10 (2023) と書籍の対応

OWASP API Security Top 10は3年毎の改訂で更新されたAPI特有の脆弱性ランキング。本書のCh.4-5はこの分類に沿って構成されている。

#	OWASP API Top 10 (2023)	書籍での扱い
API1	Broken Object Level Authorization (BOLA)	Ch.4.2-4.3
API2	Broken Authentication	Ch.4.4-4.5
API3	Broken Object Property Level Authorization	Ch.4.6 (マスアサインメント・過剰露出)
API4	Unrestricted Resource Consumption	Ch.5.1
API5	Broken Function Level Authorization (BFLA)	Ch.4.7-4.8
API6	Unrestricted Access to Sensitive Business Flows	Ch.4.9-4.10
API7	Server Side Request Forgery (SSRF)	Ch.5.2-5.3
API8	Security Misconfiguration	Ch.5.4-5.5
API9	Improper Inventory Management	Ch.5.6 (シャドウ/ゾンビ/ゴーストAPI)
API10	Unsafe Consumption of APIs	Ch.5.7-5.8

1位がBOLA、5位がBFLAであり続けている事実が、認可境界の設計の難しさを示す



Ch.4 BOLA = オブジェクトレベル認可の不備

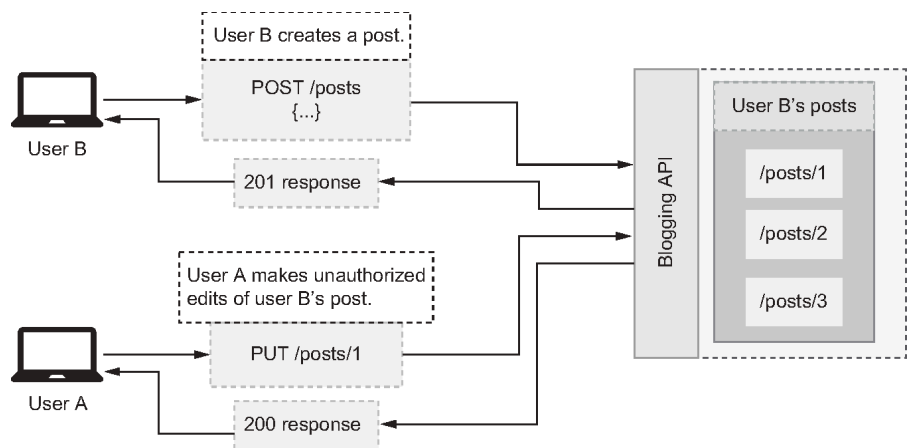


図4.1 BOLAの発生 より引用

BOLA (Broken Object Level Authorization) = ユーザーAが、ユーザーBのみがアクセスできるはずのリソース（投稿、注文、医療記録など）にアクセスできてしまう脆弱性。OWASP API Security Top 10で長らく1位。

典型的な攻撃シナリオ

```
GET /api/users/123/orders/456  ← 自分の注文（許可）
GET /api/users/124/orders/789  ← 他人の注文（本来NG）
                                ↑ ID変更だけで通ってしまう
```

防ぐためのチェックポイント

チェック項目	設計で考えること
すべての操作でリソース所有者を検証	一覧取得・詳細取得・更新・削除すべて
予測可能なIDを使わない	UUID/ULIDで列挙攻撃を困難に
認可をライブラリ任せにしない	フレームワークのデフォルトでは不十分



USPS事件――BOLAで6000万人の個人情報漏洩

2018年11月、米国郵便公社（USPS）はInformed Visibility APIの脆弱性を修正した。認証済みであれば誰でも他のユーザーのメール・電話・住所を取得できる状態だった。報告は Brian Krebs 「USPS Site Exposed Data on 60 Million Users」。

何が起きたか

段階	状況
1. 認証は通っていた	「ログインできる人」なら誰でもAPIにアクセス可能
2. 認可は機能せず	クエリで他ユーザーのデータをそのまま取得できた
3. 影響	約6000万人分のPII漏洩

この事件が示すこと

- 「認証 = 認可」ではない。ログインできる ≠ 何でも見えていい
- BOLAは医療・金融・防衛のような機密分野で侵害の主要因
- API自体は仕様通りに動いていた。脆弱性はビジネスロジックの設計欠陥

SAST/DASTでは検出されない。設計レビューと所有者検証テストでしか守れない



Ch.4 BFLA — 機能レベル認可の不備

BFLA (Broken Function Level Authorization) = 通常ユーザーが管理者用APIにアクセスできてしまう脆弱性。「URLが分からないだろう」という想定 (Security by Obscurity) に依存している実装で発生する。

典型例

POST /api/admin/users/delete ← 通常ユーザーが叩けてしまう

PUT /api/admin/config ← ロールチェックなし

防ぐ戦略

戦略	内容
アクセストークンにロール宣言	JWTのclaimsにロール情報を含める
すべてのリクエストでロール確認	ミドルウェアで一元的にチェック。各エンドポイント任せにしない
デフォルト拒否	明示的に許可されたロール以外は弾く

「URLを公開しないから安全」は幻想。攻撃者はOpenAPI仕様も漏洩トークンも入手する



Ch.4 認証の不備とマスアサインメント

認証の不備: 署名されていないトークンや期限切れトークンを受け入れる、推測可能なパスワードリセットトークンなど。**マスアサインメント:** クライアントから送られたフィールドを無条件にモデルにマッピングする問題。

マスアサインメントの例

```
// 危険 : クライアントが is_admin: true を送ると昇格してしまう
User.update(req.body)

// 安全 : 許可されたフィールドだけを抽出
User.update({ name: req.body.name, email: req.body.email })
```

関連脆弱性 : 過剰なデータ露出

APIが必要以上のフィールドを返してしまう問題。「フロントエンドで隠せばいい」と考えると、攻撃者は生のレスポンスを見る。サーバー側で必要なフィールドだけを返す。

「クライアントが信頼できる入力をくれる」前提で設計してはいけない



Ch.5 SSRF – サーバーサイドリクエストフォージェリ

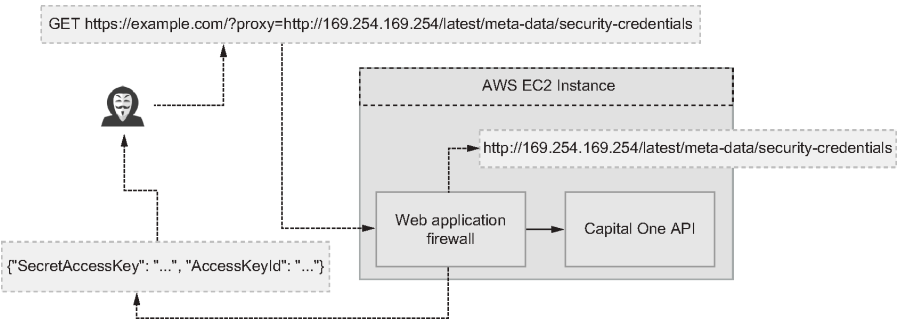


図5.5 Capital OneへのSSRF攻撃 より引用

SSRF = サーバーが外部URLを受け取って通信する機能を悪用し、攻撃者が本来アクセスできない内部ネットワークにアクセスさせる攻撃。Capital Oneの大規模侵害（2019）の根本原因。

Capital One事件の構造

WAF設定ミスを悪用 → EC2インスタンスメタデータエンドポイント（169.254.169.254）にリクエストを中継 → IAMロールの一時クレデンシャルを窃取 → AWSアクセスキー漏洩。

防御策

対策	内容
URLの厳格な検証	許可リストでドメインを限定
プライベートIPへの接続を禁止	169.254.x.x、10.x.x.x、127.x.x.xを拒否
メタデータエンドポイントの保護	IMDSv2必須化（AWSの場合）
ネットワークセグメンテーション	外部URLを叩くサービスを隔離

直近の事例――Solarman (2024) と 7pay (2019)

API認証・認可の不備は今も新規発生し続けている。直近で書籍が言及するのは Solarman事件、日本では 7pay事件。

Solarman事件 (2024年8月、Bitdefender公表)

項目	内容
対象	世界最大級の太陽光発電監視・管理プラットフォーム
脆弱性	アクセストークンの検証が不適切。攻撃者がトークンを偽造して他ユーザーに成りすまし可能
影響	個人データ窃取に加え、電力供給を妨害できる状態

7pay事件 (2019年7月、セブン&アイ)

項目	内容
対象	スマートフォン決済サービス（リリース直後）
脆弱性	大規模なアカウント乗っ取り
影響	数日間で808名・約3,860万円の被害。サービス終了に至る

「動いているからOK」ではない。トークン署名検証や認可のロジックは新規実装の最大リスクポイント



Ch.5 リソース消費・設定ミス・APIスプロール

無制限のリソース消費

ページネーションパラメータが制約なし → `?limit=100000000` でサーバーが落ちる。ブルートフォース攻撃の足場にもなる。**すべての入力に上限を設ける。**

セキュリティ設定ミス

エラーレスポンスに内部スタックトレースが漏洩、デフォルト認証情報のまま、不要なヘッダーで内部情報露出。**本番設定は別管理、エラーハンドリングは情報を最小化。**

不適切なインベントリ管理（APIスプロール）

種類	何か	リスク
シャドウAPI	把握されていないAPI	セキュリティチームが防御不可
ゾンビAPI	廃止したつもりが動いているAPI	古い脆弱性が残ったまま稼働
ゴーストAPI	ドキュメントにあるが実装されていないAPI	認証なしの隙間として悪用

「知らないAPIは守れない」。Sunsetヘッダーとオブザーバビリティでスプロールを抑制する



脆弱性は知った。次は安全に作る方法

Part 2でOWASP API Top 10の主要な脆弱性——**BOLA**、**BFLA**、**認証不備**、**マスアサインメント**、**SSRF**、**リソース消費**、**設定ミス**、**APIスプロール**——を見てきました。脅威の輪郭が見えた。

ここから先は「どう守るか」の具体です。Part 3では、**安全な設計**、**認証・認可の理論と実装**、**APIインフラ**を扱います。脆弱性を防ぐコード・アーキテクチャを作る段階に入ります。



Part 3: 設計と実装

Ch.6-9

次の問い：「安全なAPIをどう作るか？」

Ch.6 設計によるAPIセキュリティ

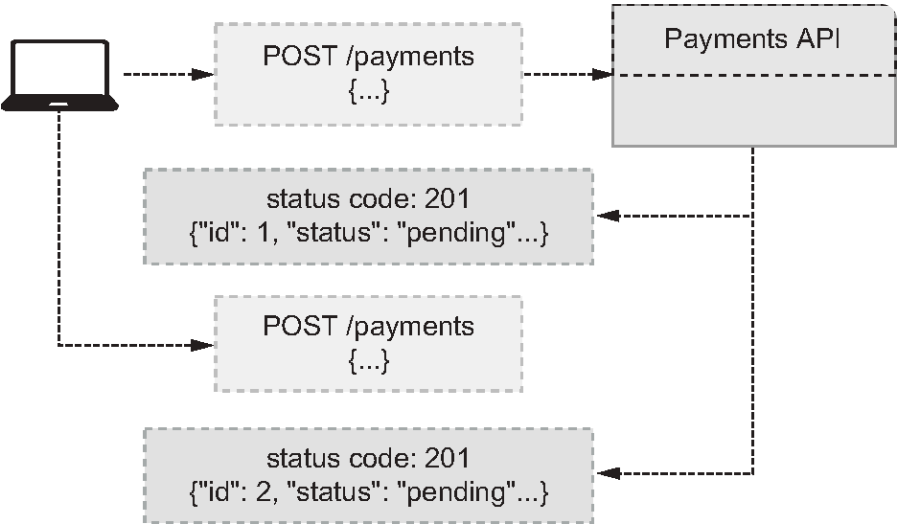


図6.4 予測可能なリソースID（増分整数ID）への列挙攻撃 より引用

セキュリティは設計フェーズから始まる。 ボルトオン型の対策は、設計の欠陥を埋められない。安全な設計は、攻撃される前提で考える。

設計時に潰すべき脆弱性パターン

パターン	問題	対策
予測可能な識別子（連番ID）	列挙攻撃でリソースが拾われる	UUID/ULIDで予測困難に
制約のないユーザー入力	巨大ペイロード、SQLインジェクション	型・長さ・列挙型でスキーマ拘束
柔軟すぎるスキーマ	サーバーが知らないフィールドも受け入れる	OpenAPIで厳格に定義、追加プロパティは禁止
サーバー側プロパティの露出	is_admin stock などの内部フィールド漏洩	入力モデルと永続化モデルを分離



OpenAPI仕様は「ドキュメント」ではなく「セキュリティ契約」。先に書け、テストで守れ

Ch.7 認証と認可の理論 — JWT・OAuth・OIDC

認証 (Authentication) = 誰であるかを確認する。**認可 (Authorization)** = 何をしてよいかを決める。両者を混同すると、「ログインできる = 何でもできる」という穴ができる。

主要技術の整理

技術	役割	注意点
JWT	署名されたトークン形式	署名アルゴリズムを必ず検証。 <code>alg: none</code> を拒否
OAuth 2.0	リソースアクセスの認可フレームワーク	認可コードフロー + PKCEを基本に
OpenID Connect	OAuthの上に構築された認証プロトコル	アクセストークンとIDトークンを混同しない
RBAC	ロールベースのアクセス制御	ロールはトークンに含めるか、認可サーバーから取得
送信者制約トークン	トークン窃取への耐性を持たせる仕組み	mTLS、DPoPで結合

「JWT使ってます」だけでは安全とは言えない。署名検証・有効期限・スコープのすべてを確認する



Ch.8 認可と認証の実装

原則は分かった。次はコードで守る。自前実装は罠だらけ。実績のあるライブラリ・OIDCプロバイダーを使うことが最初の防御線。

実装で踏み込むべき領域

実装ポイント	何をするか
OpenAPIで認証済みエンドポイント明示	<code>security</code> セクションで認証要件を文書化
JWTの発行	認可サーバーが署名・有効期限・スコープを設定
JWTの検証	署名検証、 <code>exp / iat / aud / iss</code> の確認、クロックスキュー考慮
OIDCプロバイダーとの統合	Auth0、Okta、Keycloakなど。自前認証は最後の手段
認可ミドルウェア	リクエストパスごとの認可を一元管理。ハンドラ任せにしない
RBAC実装	ロールはトークンクレームに含めるか、ポリシーエンジン（OPA）で評価

認証・認可ライブラリは「自分で書くな」。実績のあるものを使い、設定を理解する



Ch.9 安全なAPIインフラ

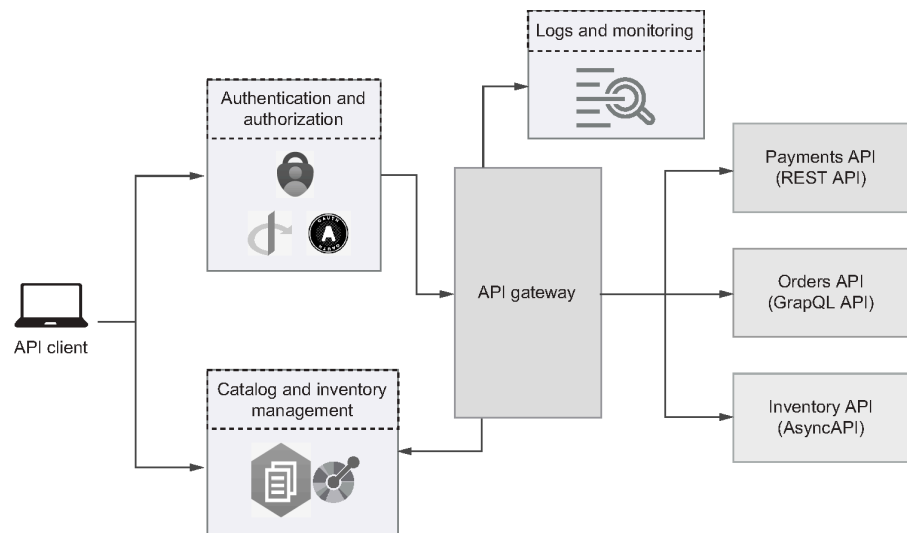


図9.1 APIゲートウェイ より引用

アプリ層で完璧でも、インフラ層が緩いと侵入される。多層防御（Defense in Depth）の考え方で、ネットワーク・ゲートウェイ・WAFを組み合わせる。

4層の防御

APIゲートウェイ

レート制限、認証、ルーティング、利用状況分析の入口。

ネットワークポロジ

機密リソースをプライベートサブネットへ。セグメンテーションで爆発半径を限定。

L3～L6の保護

DDoS対策、TLS終端、不正パケットの遮断。

WAF

L7レベルの悪意トラフィックを撃退。設定ミスは脆弱性に直結する点に注意。



各層が独立して機能することが大事。「ゲートウェイがあるからアプリは雑でもいい」は最悪

設計と実装はわかった。次は応用と運用

Part 3で安全な設計、認証・認可、インフラを見てきました。これでAPIを作る上での骨格は揃った。

しかし、金融や医療など高セキュリティ領域では追加のプロファイルが必要になる。本番運用後は、攻撃をリアルタイムに検出する仕組みが必要になる。リリース前には自動テストで守る必要がある。Part 4では、**FAPI**、**オブザーバビリティ**、**セキュリティテスト**を扱います。



Part 4: 高度な領域

Ch.10-12

最後の問い：「運用しながらどう守り続けるか？」

OAuth 2.0認可コードフロー + PKCE

OAuth 2.0で「正しい」と推奨されるのは、認可コードフロー + PKCE。モバイル・SPAでも、サーバーサイドWebでも、これが基本形。Implicit Flowは廃止された。

フローの要点

ステップ	何が起こる
1. クライアントが <code>code_verifier</code> を生成	ランダム文字列。後で証明として送る
2. <code>code_challenge</code> (verifierのハッシュ)を認可リクエストに添付	中間者が認可コードを盗んでも使えなくなる
3. ユーザー認証 → 認可サーバーが <code>code</code> 発行	URLパラメータで返ってくる
4. クライアントが <code>code</code> + <code>code_verifier</code> をトークンエンドポイントへ	認可サーバーがハッシュ照合
5. アクセストークン (+ リフレッシュトークン) 発行	スコープと有効期限が含まれる

関連プロトコル

OpenID Connect (OIDC)は OAuth 2.0 の上に**認証**を載せる。 `id_token` (JWT) でユーザーの身元情報を取得。「**OAuth = 認可、OIDC = 認証**」として整理する。

RFC 9700 (2024) 以降、PKCEはconfidential clientでも必須推奨。「公開クライアントだけ」という旧理解を更新する



Ch.10 金融グレードAPI (FAPI)

FAPI = Financial-grade API。OpenID Foundationが策定した、金融機関レベルのセキュリティを要求するAPI向けのプロファイル。オープンバンキング、決済APIなど高リスク領域で採用。

FAPI 2.0が要求する強化

領域	標準OAuthとの違い
認可リクエスト	PARでパラメータをサーバー側に保護。改ざん耐性を強化
トークン	送信者制約（mTLSまたはDPoP）でトークン窃取に耐性
メッセージ署名	リクエスト・レスポンスをJWSで署名し、改ざん検出
クライアント認証	private_key_jwt または mTLS。client_secret_basicは禁止

FAPIは金融のためだけではない。「高い保証が必要なAPI」全般のリファレンスとして使える



送信者制約トークン——mTLS と DPoP

OAuth 2.1の仕様に推奨される、トークン窃取への耐性を持たせる仕組み。Bearer Tokenは「持っている人＝使える人」だが、送信者制約トークンは「正当な所有者だけ使える」。FAPI 2.0でも必須要件。

2つの実装方式

方式	仕組み	適用領域
mTLS	TLSハンドシェイクの <code>CertificateVerify</code> 段階でクライアント証明書による署名。トークンはその証明書にバインド	サービス間（B2B、内部マイクロサービス）に強い
DPoP	クライアントの公開鍵をリクエストごとにJWTで署名して提示。証明書管理を省略可能	パブリッククライアント（モバイル・SPA）に向く

Bearer Token との違い

```
[Bearer] Authorization: Bearer eyJhbGc ...    ← 持参すれば誰でも通る
[DPoP]   Authorization: DPoP eyJhbGc ...
          DPoP: <signed JWT proving private-key possession>    ← 鍵を持つ者だけ通る
```

「漏洩したトークンが悪用できない」状態を作る。これがリフレッシュトークンの再利用攻撃にも効く



FAPI 2.0 vs 標準OAuth 2.0 の差分

FAPIは英国Open Banking、ブラジルOpen Banking、豪州Consumer Data Rights Standardsで採用されている。金融以外でも「高い保証が必要なAPI」のリファレンスとして使える。

標準OAuthからの主な強化点

領域	OAuth 2.0 (RFC 6749)	FAPI 2.0 セキュリティプロファイル
認可リクエスト	URLパラメータでパススルー	PAR必須 (Pushed Authorization Requests) でリクエストをサーバー側に保護
クライアント認証	<code>client_secret_basic</code> も可	private_key_jwt or mTLS のみ 。共有秘密の禁止
トークン保護	Bearer Token (持参すれば使える)	送信者制約 (mTLS or DPoP) でトークン窃取に耐性
メッセージ完全性	TLSのみに依存	JWS署名 でリクエスト・レスポンスを保護 (メッセージ署名プロファイル)
クロスサイト	パラメータ改ざんの余地	PARで認可パラメータが改ざんできない

FAPIは「オープンバンキング API のため」だが、ヘルスケア・公共サービスなど高保証領域にも展開可能



Ch.11 APIセキュリティのためのオブザーバビリティ

侵入を防ぐだけでは不十分。「何が起きているか」を見続け、攻撃の兆候を検出する。通常のオブザーバビリティ（ログ・トレース・メトリクス）に、セキュリティの観点を加える。

検出すべきパターン

攻撃パターン	検出のヒント
入力ベースの攻撃	異常な文字列パターン、エラー率の急増
エンドポイント乱用	レート異常、botに特徴的な間隔（ms単位）でのリクエスト
認証情報の総当たり	同一IPからの大量4xx、ユーザー単位の失敗集中
BOLA試行	認可されていないIDへの大量アクセス試行
データ流出	通常より大きなレスポンスサイズ、機密エンドポイントへの集中

セキュリティイベントは独立したログにせず、通常の観測基盤に統合する。事象を関連付けやすくなる



Ch.12 APIセキュリティのテスト

セキュリティを「リリース前の手動チェック」から「CI/CDの自動テスト」に組み込む。設計欠陥は手動レビューが、実装欠陥は自動テストが向く。両方を組み合わせる。

テスト戦略の階層

テスト種別	何を検出するか
コントラクトテスト	OpenAPI仕様との実装乖離（スプロール・ゴーストAPI予防）
ファジング	想定外の入力でのクラッシュ・例外漏洩
アクセス制御テスト（自動化）	BOLA・BFLAをユーザー切り替えで検証
ビジネスフロー脆弱性テスト	多段ユーザーフローの順序バイパス、レース条件
設計レビュー（手動）	認可境界、データモデル、信頼境界

「リリース前にやる」ではなく「コミットごとに走る」。テストの自動化はシフトレフトの実装



APIドリフトを潰すーコントラクトテストとファジング

APIドリフト = API仕様（OpenAPI）と実装の乖離。多くのAPI侵害の根本原因はここに潜む。

ドリフトが生む2種類のリスク

種別	何が起きるか
オーファンエンドポイント	仕様にあるが未実装。 404 を返す。あるべきAPIが無い
シャドウエンドポイント	実装にあるが仕様に無い。攻撃者の侵入口（OWASP API9）

主要ツール

ツール	特徴
Schemathesis (Python)	OpenAPIから動的にテストケースを生成。ファジングを兼ねる
Microcks	コンシューマー側のコントラクトテストもサポート
Dredd / Restler-fuzzer / API-fuzzer	用途別の選択肢

ファジングが見つけるもの

不正な形式の入力（巨大文字列、型違反、特殊文字）を投げて、クラッシュ・例外漏洩・想定外のステータスを検出。攻撃者が同じことを試す前にCIで潰す。



OpenAPIは「ドキュメント」ではなく「契約」。コントラクトテストでドリフトを継続的に潰す

書籍全体のメッセージ

12章を通じて書籍が一貫して伝えているのは、APIセキュリティは「認証・認可の話」ではなく「設計・実装・インフラを貫く包括的な実践」であるということ。

3つの原則

原則	意味
セキュア・バイ・デザイン	設計段階からセキュリティを組み込む
シフトレフト	開発サイクルの早い段階で検証する
ゼロトラスト	すべてを検証する。内部APIも例外ではない

セキュリティを後付けするな。設計に織り込み、コードで守り、運用で見続けよ。



どこから読むか

本音を言えば、Ch.1からCh.12まで順番に読んでほしい。脆弱性を知ってから設計・実装に入る流れが最も学びが深くなる。

ただ、目的別に以下のルートからでも読み始められます。

脆弱性を理解したい

Ch.4 → Ch.5 → Ch.6

OWASP Top 10 → 設定・SSRF →
設計で潰す

実装中で困っている

Ch.7 → Ch.8 → Ch.9

JWT/OAuth/OIDC → 実装 → インフ
ラ

運用で守りたい

Ch.11 → Ch.12 → Ch.3

オブザーバビリティ → テスト → ゼ
ロトラスト



本日のまとめ

書籍の全体像: 原則 → 脆弱性 → 設計と実装 → 高度な領域の4部・12章。設計・実装・インフラの3軸でAPIを守り、シフトレフト・ゼロトラスト・セキュア・バイ・デザインを実践する。

翻訳者として一番伝えたいこと

API侵害の多くはビジネスロジックの欠陥から起きる。SSL証明書を入れ、WAFを置き、JWT認証を導入しても、**BOLA**や**BFLA**のような「認可境界の設計ミス」は防げない。チェックリストにチェックを打つだけでは守れない。守るのは設計判断とコードレビューだ。「設計で潰したか」「実装で検証したか」「運用で観測しているか」——この3つの問いを離さないこと。

「全部読む時間がない」への回答

Ch.3（原則）、Ch.4（OWASP脆弱性）、Ch.6（設計）——この3章を順に読めば、設計レビューに必要な視点は手に入る。Ch.4だけだと「攻撃は知っているが防ぎ方が断片的」、Ch.6だけだと「正しく作る前提が抜ける」。**Ch.3 → Ch.4 → Ch.6** の順で読むのが最短経路。

APIを公開する瞬間にあなたはドアを開けている。鍵をかけ続けるのは作る人の責任だ。



参考資料

- [Secure APIs](#) - Jose Haro Peralta (Manning)
- [Secure APIs 日本語版](#) - 株式会社スリーシェイク訳
- [OWASP API Security Top 10](#) - 2023版
- [OWASP API Security Top 10 日本語訳](#)
- [NIST SP 800-207 Zero Trust Architecture](#)
- [RFC 6749 OAuth 2.0 / 日本語訳](#)
- [JWT.IO 日本語](#)
- [FAPI 2.0 Security Profile](#) - OpenID Foundation
- [API-Security-Checklist](#) - Shieldfy
- [HackerOne](#) - バグバウンティプラットフォーム
- [デジタル庁 APIテクニカルガイドブック](#)



The logo for 3 SHAKE, featuring the number '3' followed by a small icon of three overlapping colored triangles (yellow, green, blue) and the word 'SHAKE' in a bold, white, sans-serif font.

3 SHAKE

ありがとうございました

ご質問・ご相談はお気軽にお問い合わせください

@nwiizo | <https://3-shake.com>